HintonMarket System

Deliverable 2: System Decomposition and Database Integration

Team 55

Hussain Almahooz – 101258192

Khalil Bruce – 101308601

Shaswat Tyagi – 101263508

Paimana Sultani – 101287827

Submitted to:
Dr. Christine Laurendeau
COMP 3004 - Object-Oriented Software Engineering
Winter 2026
School of Computer Science
Carleton University

April 3rd, 2026

Table of Contents

Table of Contents 2

Section 1: D1 System Decomposition Diagram 3

Section 2: D2 System Decomposition Diagram 4

Section 3: Explanation of Design Changes 5

3.1 What Changed 5

3.2 Impact on Cohesion and Coupling 6

3.3 Architecture 7

Section 1: D1 System Decomposition Diagram

Figure 1: HintonMarket Deliverable 1 – System Decomposition Class Diagram Showing Subsystem Organization and Dependencies

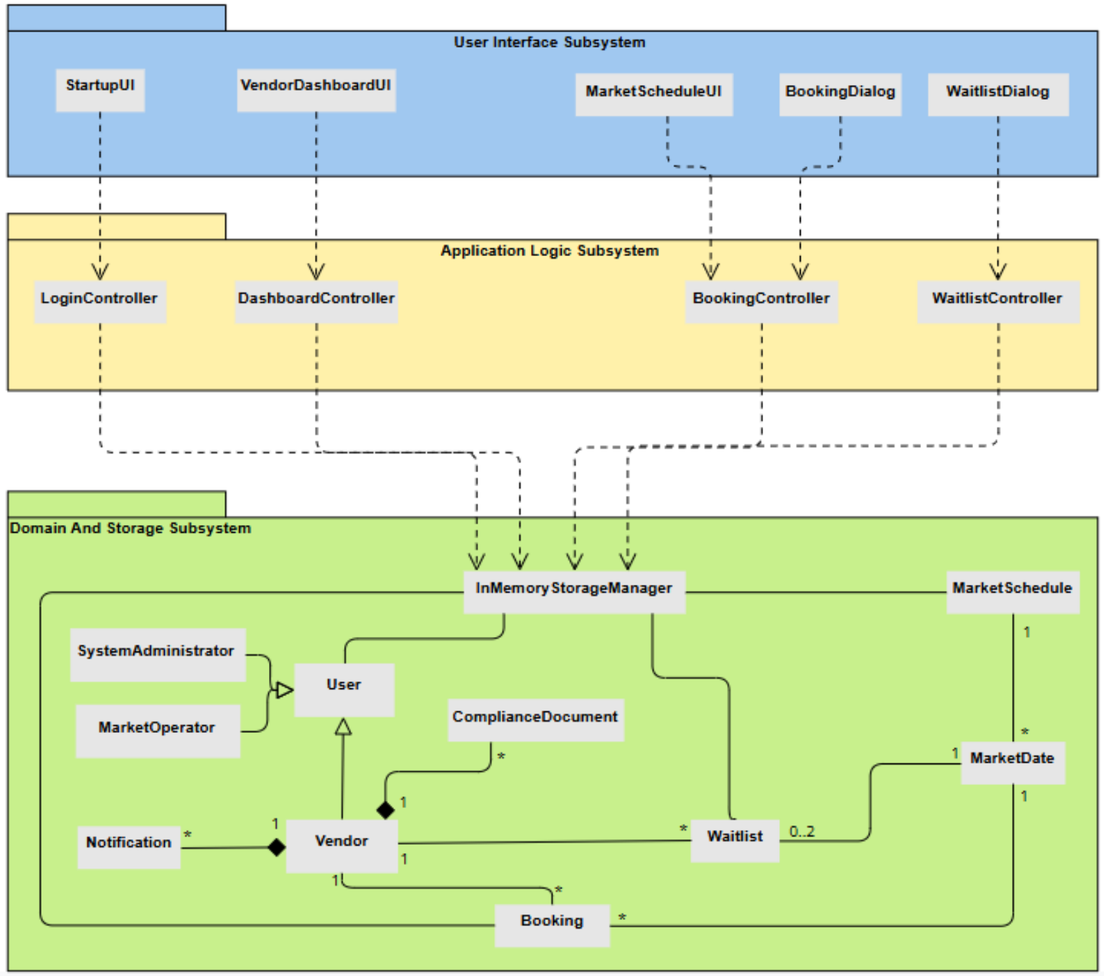

Figure 1 illustrates the subsystem decomposition of the HintonMarket system for Deliverable 1. The diagram shows the separation between the User Interface subsystem, Application Logic subsystem, and Domain and Storage subsystem, as well as the relationships among the domain entities used in the implemented vendor workflow. It also includes a general User abstraction to support the startup identification for different user types.

Section 2: D2 System Decomposition Diagram

Figure 2: HintonMarket Deliverable 2 – System Decomposition Class Diagram with Persistence and Market Operator Support

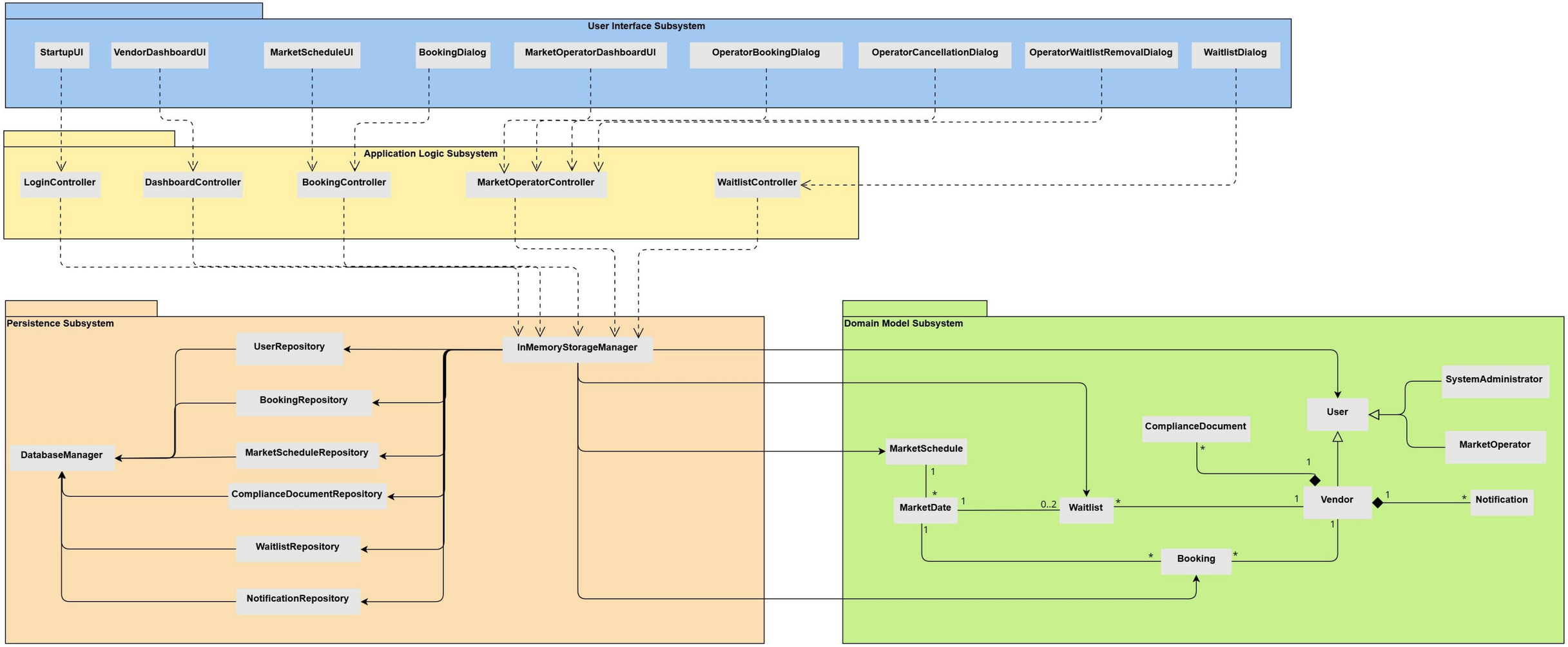

Figure 2 illustrates the subsystem decomposition of the HintonMarket system for Deliverable 2. The diagram shows the separation between the User Interface subsystem, Application Logic subsystem, Persistence subsystem, and Domain Model subsystem, as well as the relationships among the major classes used in the updated implementation. It reflects the transition from the Deliverable 1 in memory design to a structure that supports SQLite persistence, repository-based data access, in memory runtime object management, and the addition of Market Operator functionality.

Section 3: Explanation of Design Changes

3.1 What Changed

The Deliverable 2 system decomposition builds on the Deliverable 1 design by reorganizing the system to support persistent storage and additional market operator functionality. In Deliverable 1, the system was structured around three main subsystems: User Interface, Application Logic, and Domain and Storage. Controllers interacted primarily with the InMemoryStorageManager, which served as the central class for storing and retrieving objects in memory.

In Deliverable 2, the design was expanded into four subsystems: User Interface, Application Logic, Persistence, and Domain Model. The User Interface and Application Logic subsystems were extended to support the additional market operator features required in Deliverable 2, including booking a stall on behalf of a vendor, cancelling a vendor booking, and removing a vendor from a waitlist. The User Interface subsystem was also expanded to include dedicated market operator dialog classes for these actions.

The most significant structural change was the introduction of a dedicated Persistence subsystem. Instead of relying only on a centralized in-memory storage class, the updated design now includes DatabaseManager and repository classes such as UserRepository, MarketScheduleRepository, BookingRepository, WaitlistRepository, NotificationRepository, and ComplianceDocumentRepository. This change reflects the Deliverable 2 requirement that data must be stored persistently in SQLite and also recreated as in-memory objects during execution.

Although InMemoryStorageManager remains part of the implementation, its role has changed from being the main storage mechanism in Deliverable 1 to acting as a runtime in-memory object manager that works with the repository classes to load persisted data from the database into domain objects. This is reflected in the code, where InMemoryStorageManager::loadFromDatabase() creates repository objects, retrieves persisted records, and reconstructs users, market dates, waitlists, bookings, and compliance documents in memory.

The Domain Model subsystem was also clarified. In Deliverable 1, domain classes and storage responsibilities were grouped together in the same subsystem. In Deliverable 2, those responsibilities were separated more clearly by keeping the business entities in the Domain Model subsystem and moving technical data access responsibilities into the Persistence subsystem. This makes the updated design more aligned with the requirements of SQLite persistence.

3.2 Impact on Cohesion and Coupling

The design changes made in Deliverable 2 improved both cohesion and coupling when compared with Deliverable 1. In the original design, the InMemoryStorageManager was responsible for centralized in-memory data access, and domain and storage concerns were grouped together in one subsystem. While this worked for the D1 prototype, it mixed business entities with technical storage responsibilities.

In the Deliverable 2 design, cohesion was improved by assigning more focused responsibilities to each subsystem. The User Interface subsystem remains responsible only for presentation and user interaction. The Application Logic subsystem coordinates use-case behavior through controller classes. The new Persistence subsystem is responsible for database access, repository coordination, and loading persisted data into runtime memory structures. The Domain Model subsystem is now focused on representing the core business concepts of the HintonMarket system, such as users, bookings, waitlists, market dates, and compliance documents. This clearer separation of responsibilities makes each subsystem more internally consistent and easier to understand and maintain.

Coupling was also improved because the application logic no longer needs to depend directly on low-level database operations. The DatabaseManager encapsulates the SQLite connection, while the repository classes handle data retrieval and storage. InMemoryStorageManager then coordinates the loading of that persistent data into objects used by the running system. As a result, controllers remain separated from direct database access and continue to work through a structured intermediate layer. This reduces the impact of database-specific changes on the rest of the application and makes the design more modular.

Another improvement is that the updated design supports persistence without forcing the domain entities themselves to depend on database code. The business classes remain focused on representing market concepts and relationships, while persistence concerns are handled elsewhere. This is an improvement over the Deliverable 1 structure because it preserves the object-oriented domain model while still satisfying the Deliverable 2 requirement that all data be stored both in SQLite and in memory.

The Deliverable 2 changes improved cohesion by giving each subsystem a more focused responsibility, and improved coupling by separating controllers and domain entities from low-level persistence details.

3.3 Architecture

The updated Deliverable 2 design is based on a layered object-oriented architecture that follows a Boundary-Control-Entity style, extended with a dedicated Persistence layer. The User Interface subsystem acts as the boundary layer, the Application Logic subsystem acts as the control layer, and the Domain Model subsystem contains the main entity classes. Compared with Deliverable 1, the design now includes a distinct Persistence subsystem to support SQLite-based storage and retrieval.

In Deliverable 1, the architecture was layered around user interface classes, controllers, and a combined domain/storage subsystem centered on the InMemoryStorageManager. In Deliverable 2, the architecture evolved to better support persistence requirements. The system now follows the general flow of user interface classes sending requests to controllers, controllers interacting with InMemoryStorageManager, InMemoryStorageManager coordinating with repository classes, and repositories using DatabaseManager to access SQLite. Persisted data is then reconstructed into in-memory domain objects for use during runtime.

This means the overall architectural style remained layered, but it was refined to include repository-based persistence and clearer separation between technical infrastructure and domain logic. This revised architecture better supports the persistence requirements of Deliverable 2.